

Project Report

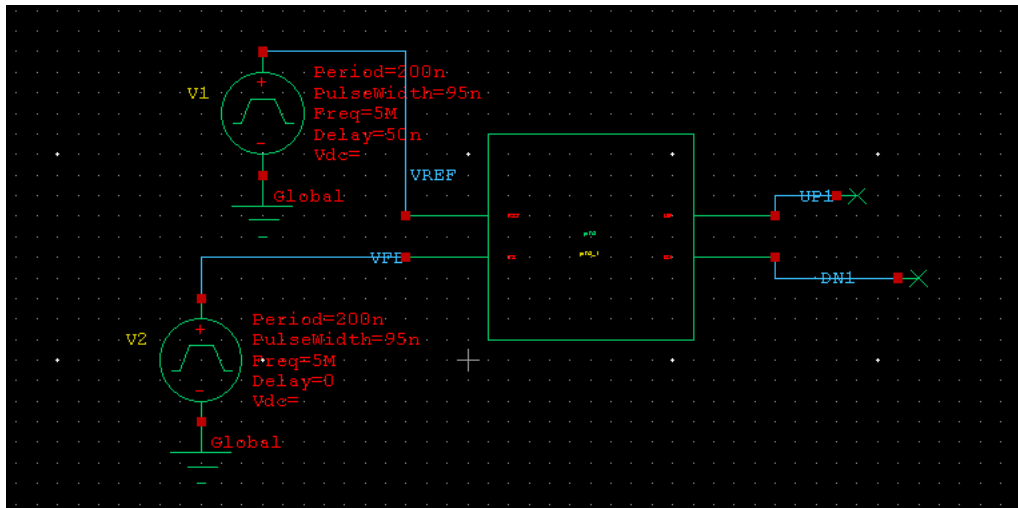
PLL Design and Verilog-A Behavioral Modeling

1. Verilog-A model of the PFD:

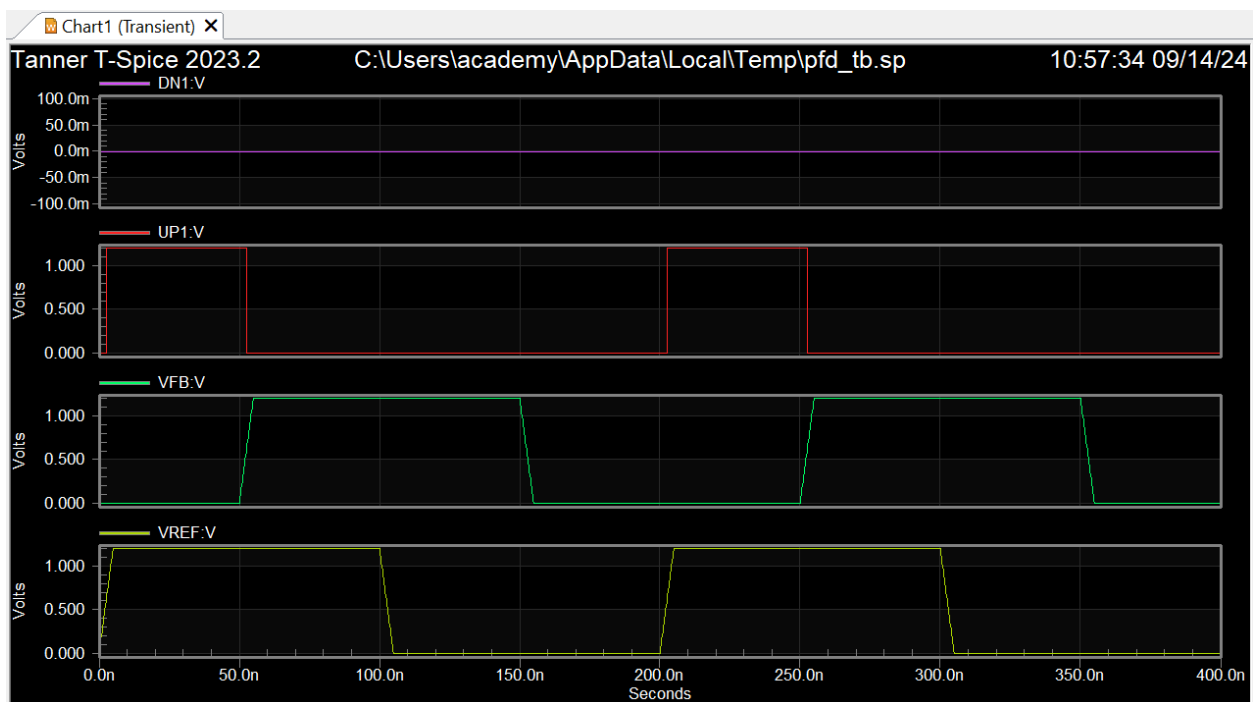
```
PFD pfd:verilog* pfd_tb/pfd_1 X
// AMS PLL Project: Phase Frequency Detector (PFD)
`include "constants.vams"
`include "disciplines.vams"
// REF: Reference signal
// FB: Feedback signal
// UP: Up signal (FB late)
// DN: Down signal (FB early)
module pfd(REF,FB,UP,DN);
    // VDD and threshold voltage for digital signals
    parameter real VDD = 1.2;
    parameter real thresh = 0.6;
    // rise/fall/delay times of PFD output
    parameter real trise = 10p, tfall = 10p, td = 0;
    input REF,FB;
    output UP,DN;
    electrical REF,FB,UP,DN;
    // Internal UP and DN signals
    real UP_i,DN_i ;
    analog begin
```

```
PFD pfd:verilog* pfd_tb/pfd_1 X
// Check DN_i state when REF arrives
@(cross(V(REF)-thresh,1))
    if(DN_i < thresh)
        UP_i = VDD;
    else begin
        UP_i = 0;
        DN_i = 0;
    end
// Check UP_i state when FB arrives
@(cross(V(FB)-thresh,1))
    // *** add line here ***
    if(UP_i < thresh)
        DN_i = VDD;
    else begin
        UP_i = 0;
        DN_i = 0;
    end
    V(UP) <+ transition(UP_i,td,trise,tfall);
    V(DN) <+ transition(DN_i,td,trise,tfall);
end
endmodule
```

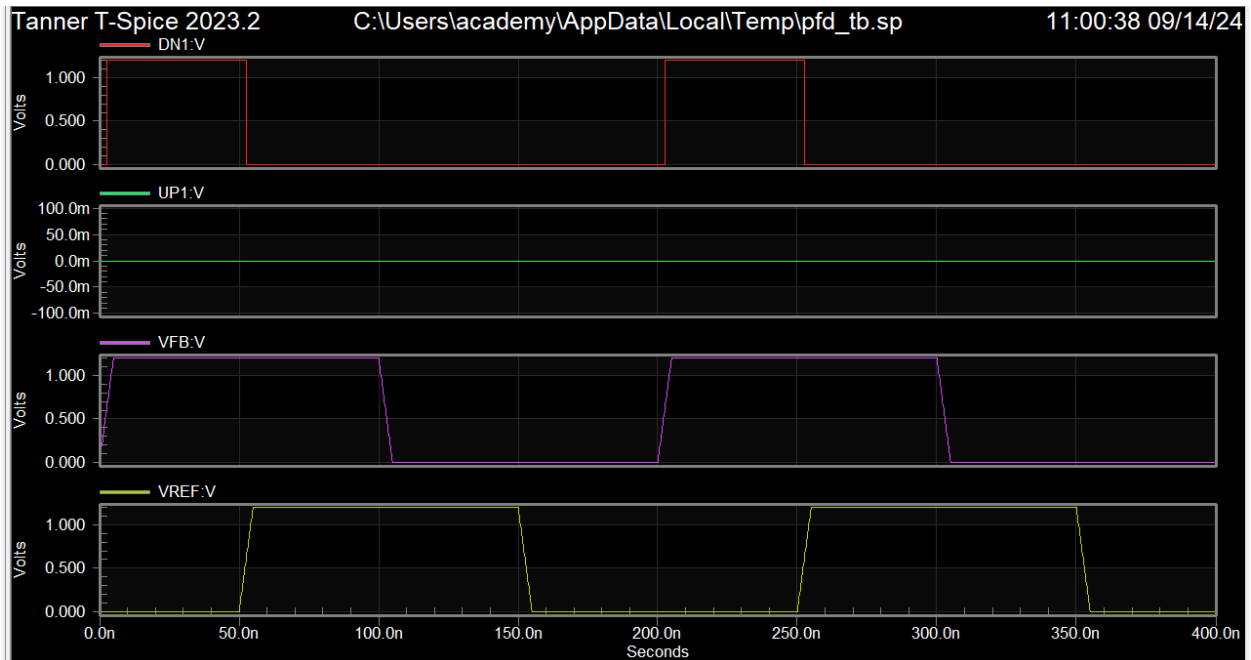
2. PFD testbench and results:



When VREF is leading



When VFB is leading:



3. Verilog-A model for the CHP:

```
// AMS PLL Project: Charge Pump (CHP)

`include "constants.vams"
`include "disciplines.vams"

// UP: Up signal
// DN: Dn signal
// IOUT: CHP current output
// Since the output is current, IOUT cannot be left unconnected (o.c.) in the testbench
module CHP(UP,DN,IOUT);
    input UP,DN;
    inout IOUT;
    electrical UP,DN,IOUT;

    // ichp: CHP current
    parameter real ichp = 10u from [0:inf];
    // Threshold voltage for digital signals
    parameter real thresh=0.6;
    // rise/fall/delay times of CHP output
    parameter real trise=10p, tfall=10p, td=0;
```

```

PFD CHP:verilog X
// Internal variable for CHP output current
real IOUT_i = 0;

analog begin

    // Generate events at UP and DN transitions
    @(cross(V(UP)-thresh,0))
    ;
    @(cross(V(DN)-thresh,0))
    ;

    if ((V(UP) > thresh) && (V(DN) < thresh))
        IOUT_i = -ichp;
    else if ((V(DN) > thresh) && (V(UP) < thresh))
        IOUT_i = ichp;
    else
        IOUT_i=0;

    I(IOUT) <+ transition(IOUT_i,td,trise,tfall);

end

```

```

PFD CHP:verilog X

// Generate events at UP and DN transitions
@(cross(V(UP)-thresh,0))
;
@(cross(V(DN)-thresh,0))
;

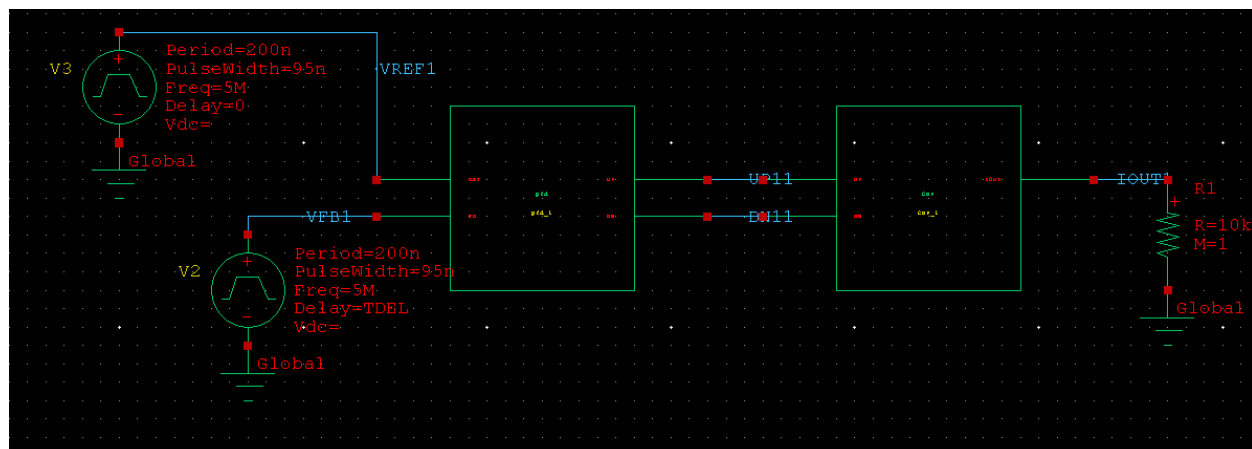
if ((V(UP) > thresh) && (V(DN) < thresh))
    IOUT_i = -ichp;
else if ((V(DN) > thresh) && (V(UP) < thresh))
    IOUT_i = ichp;
else
    IOUT_i=0;

I(IOUT) <+ transition(IOUT_i,td,trise,tfall);

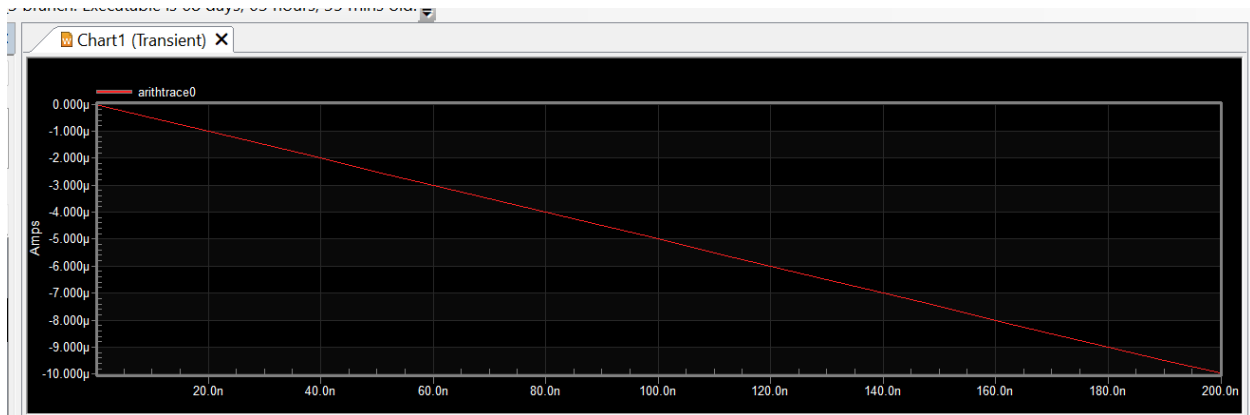
end
endmodule

```

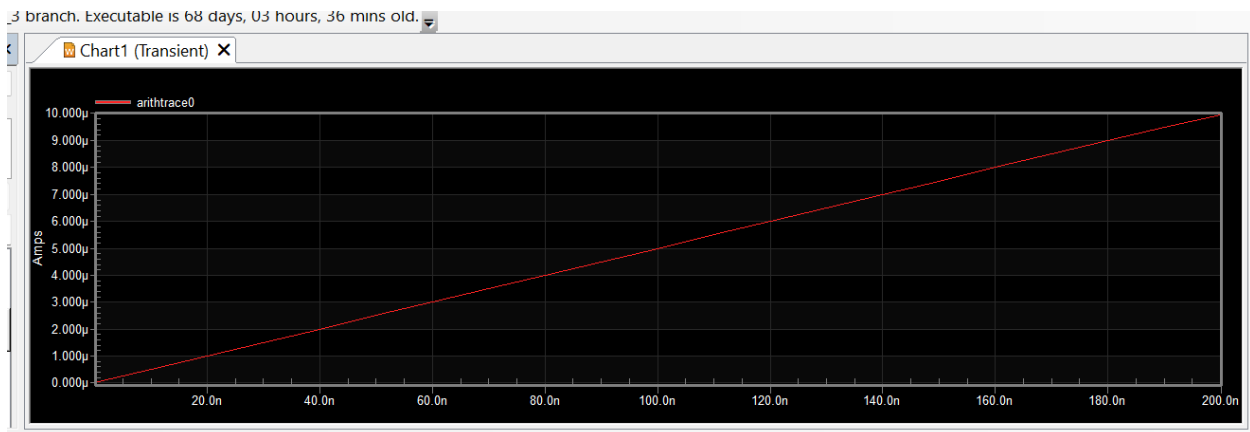
4. CHP testbench and results:



Sweep on VREF delay:



Sweep on VFB delay:



5. Your Verilog-A model for the VCO:

From random number generator: $kvco = 2.1 \times 10^9 \text{ Hz/V}$

210

Lower Limit

Upper Limit

Generate  **Clear**

```

// AMS PLL Project: Voltage Controlled Oscillator (VCO)

`include "constants.vams"
`include "disciplines.vams"

module VCO(VCTRL,VOUT);

    parameter real VHIGH = 1.2;
    parameter real Vmin=0.2;
    parameter real Vmax=1 from (Vmin:inf);
    parameter real Fmin=0.5G from (0:inf);
    parameter real Fmax=1.5G from (Fmin:inf);
    parameter real trise=10p, tfall=10p, td=0;

    input VCTRL;
    output VOUT;
    voltage VCTRL,VOUT;

    real freq,phase,sine,VOUT_i;

```

VCO:veriloga X

```

analog begin

    // compute the freq from the input voltage
    freq = ((V(VCTRL) - Vmin)*2.1*1e9) + Fmin;
    // bound the frequency
    if (freq > Fmax) freq = Fmax;
    if (freq < Fmin) freq = Fmin;

    // calculate the phase (modulo 2*pi)
    phase=2*`M_PI*idtmod(freq,0.0,1.0,-0.5);

    // generate the output
    sine = sin(phase);

    @(cross(sine,0))
        ;

    if (sine > 0)
        VOUT_i=VHIGH;
    else
        VOUT_i=0.

```

```

VCO:verilog
sine = sin(phase);

@(cross(sine,0))
;

if (sine > 0)
    VOUT_i=VHIGH;
else
    VOUT_i=0;

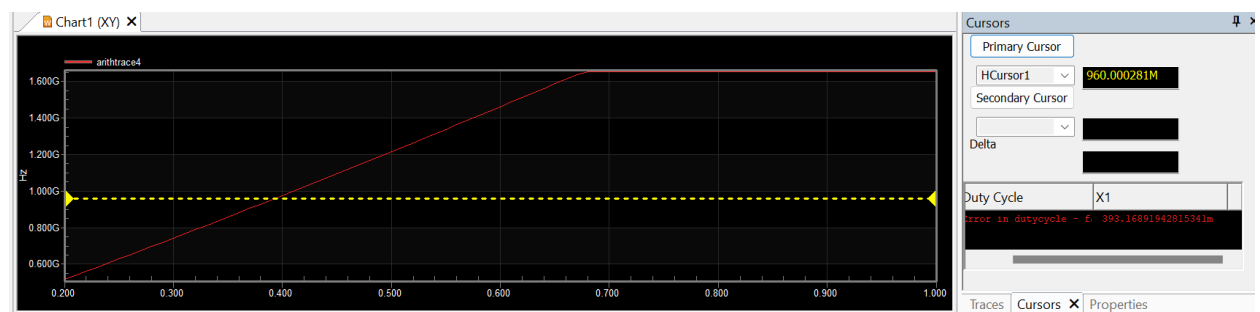
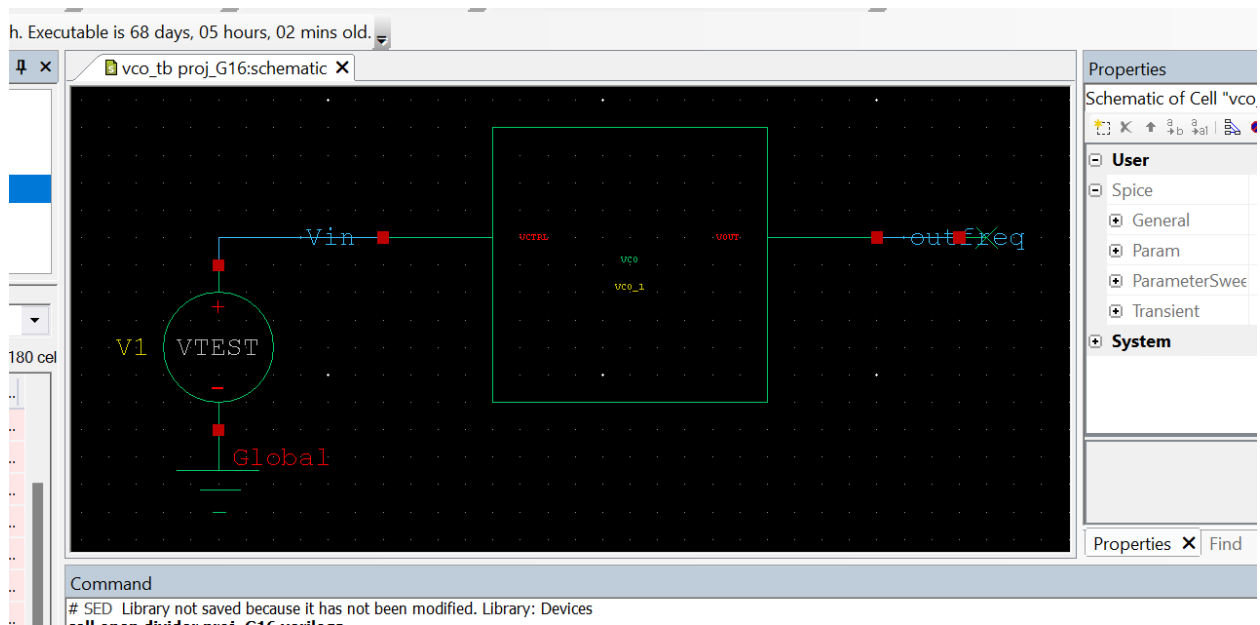
V(VOUT) <+ transition(VOUT_i,td,trise,tfall);

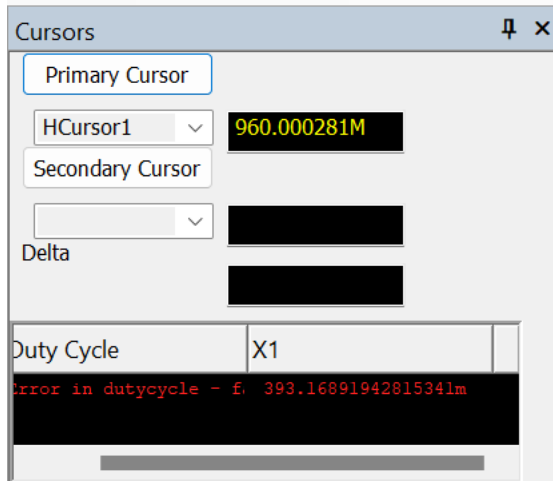
// bound the time step
$bound_step(0.1/freq);

end
endmodule

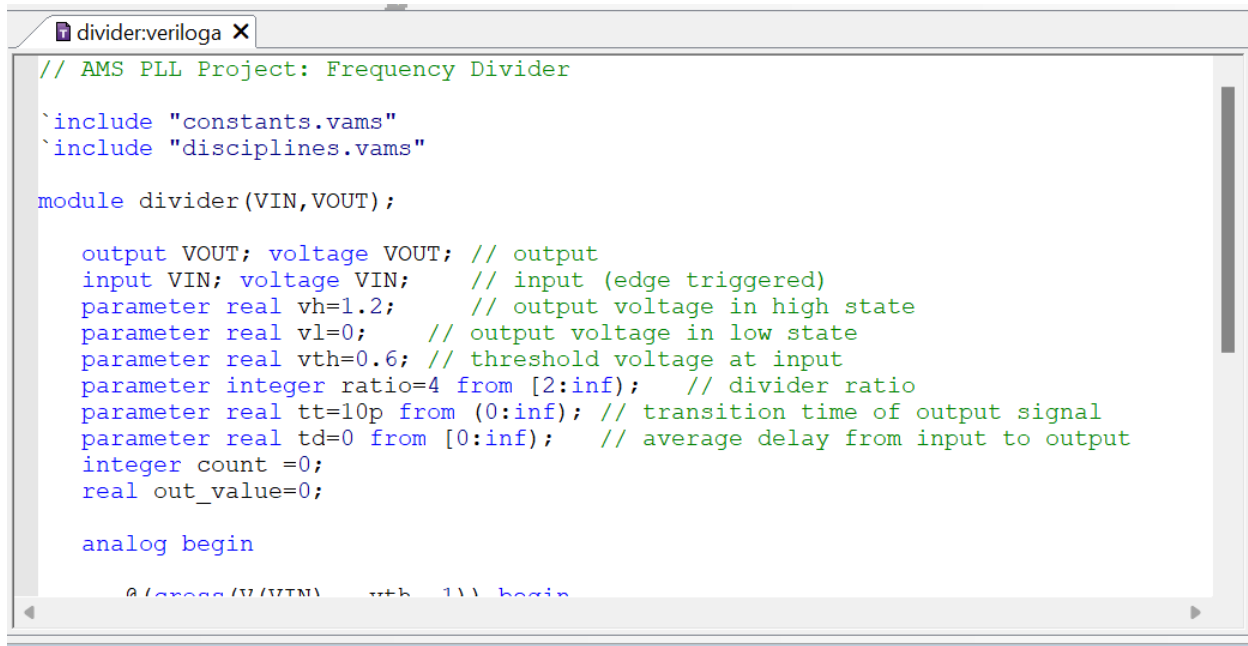
```

6. VCO testbench and results:





7. Verilog-A model for the divider:



```

analog begin

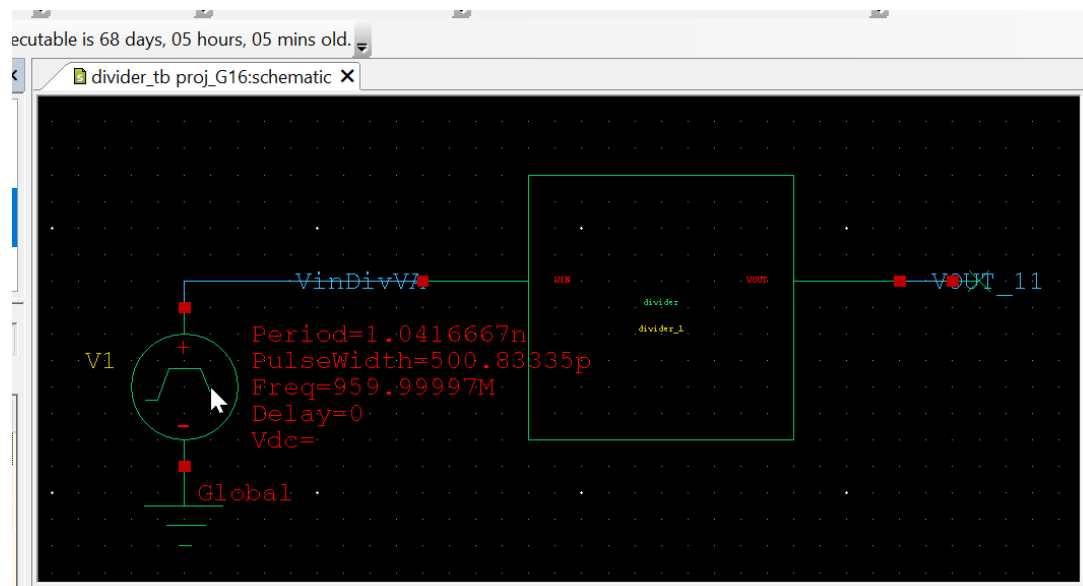
    @(cross(V(VIN) - vth, 1)) begin
        if (count==floor(ratio/2))
            out_value = vh;
            // *** add line here ***
        else if(count==0 || count==ratio) begin
            out_value = vl;
            count=0;
        end
        count = count + 1;
    end

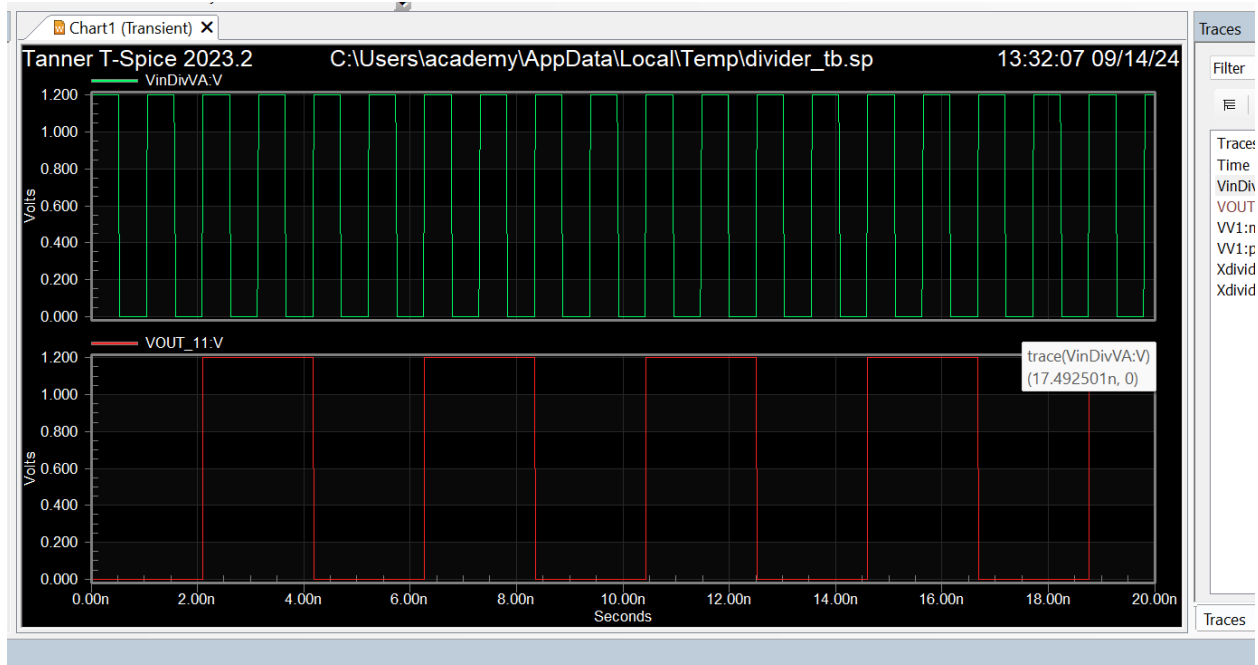
    V(VOUT) <+ transition(out_value, td, tt);

end
endmodule

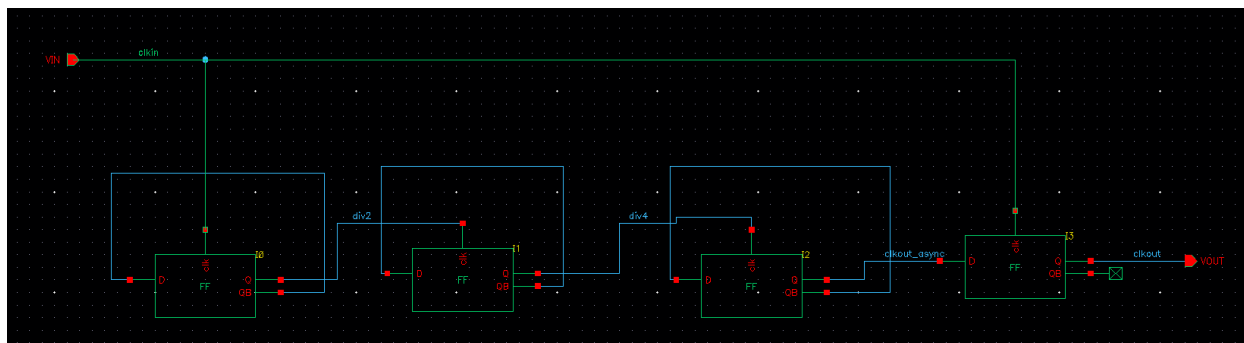
```

8. Divider testbench and results:

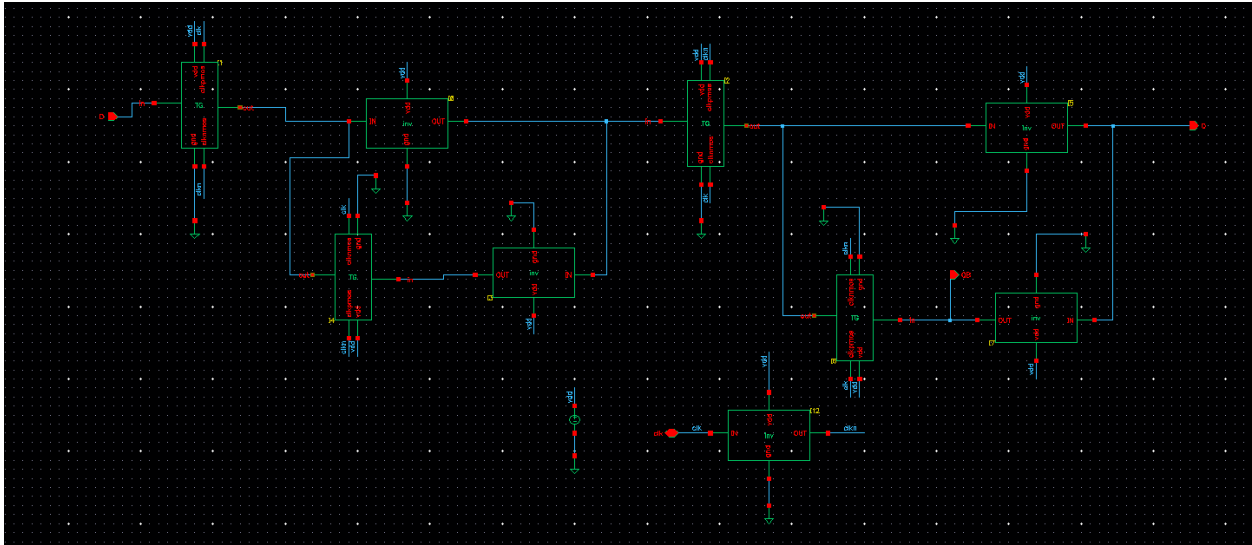




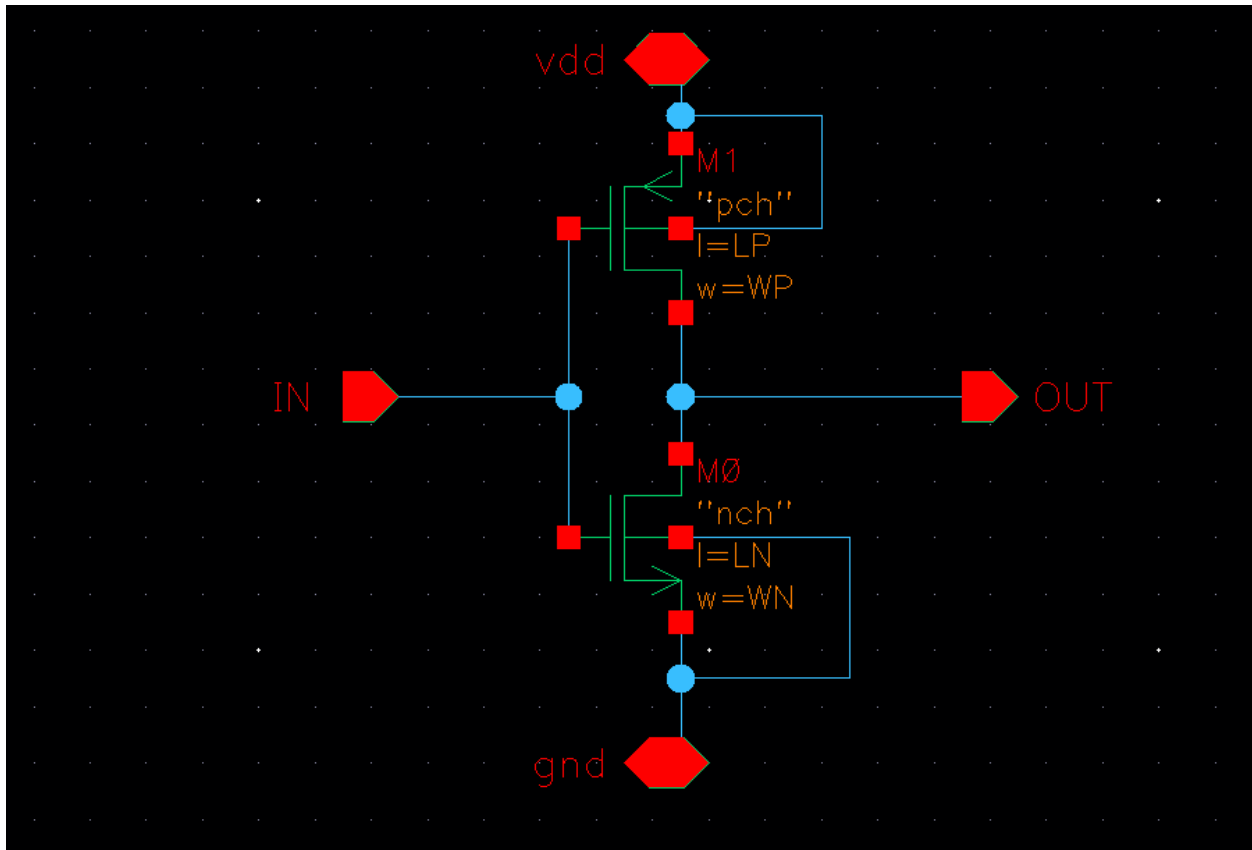
9. Transistor level divider ($LN=LP=180\text{nm}$, $WN=1\mu\text{m}$, $WP=2\mu\text{m}$):



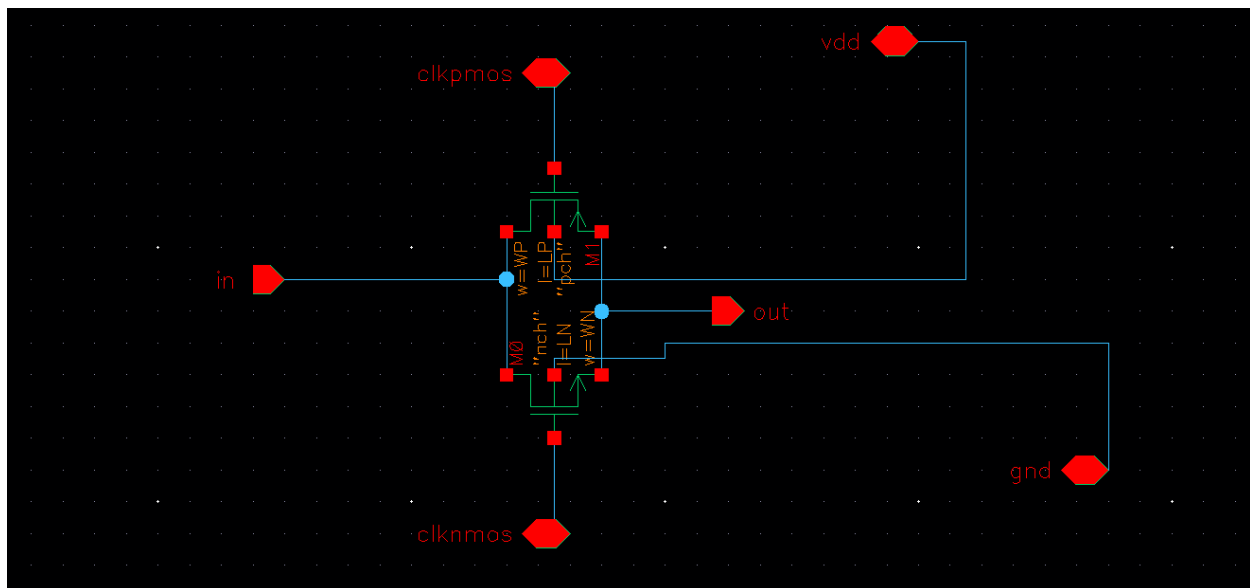
DFF:



Inverter (inv):



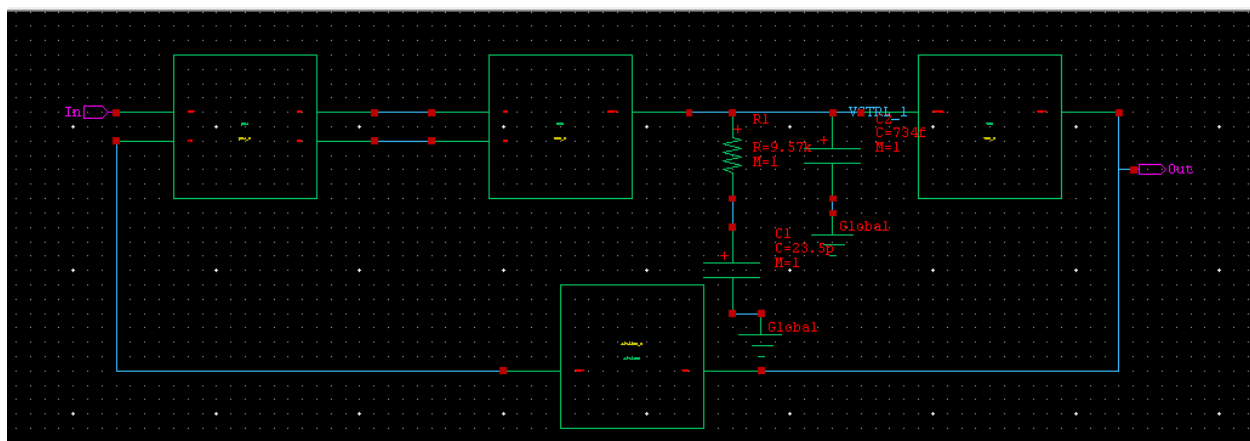
TG:



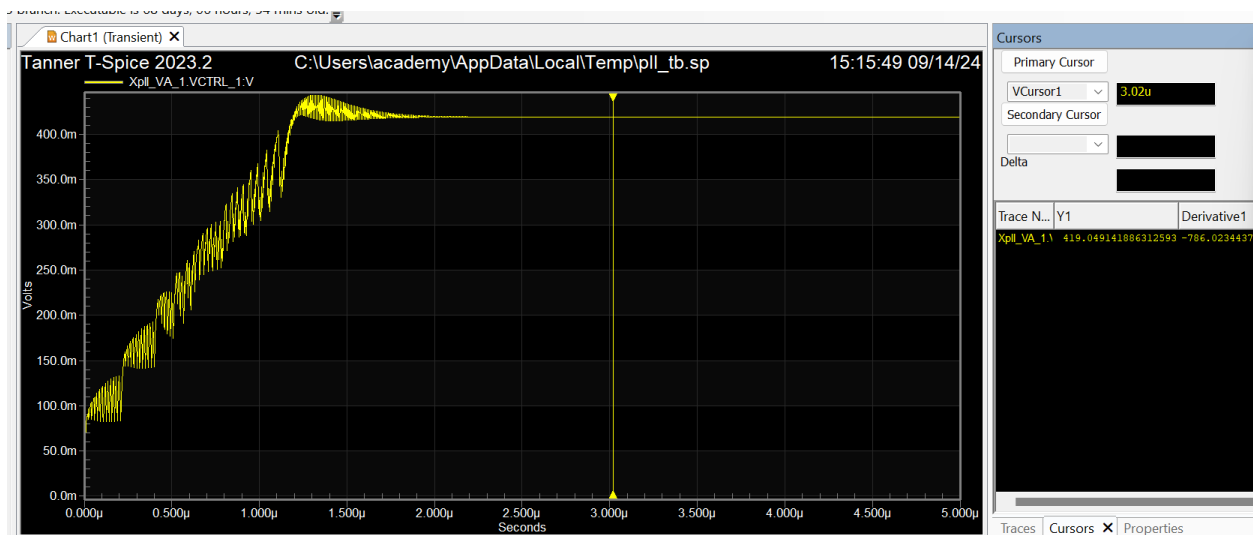
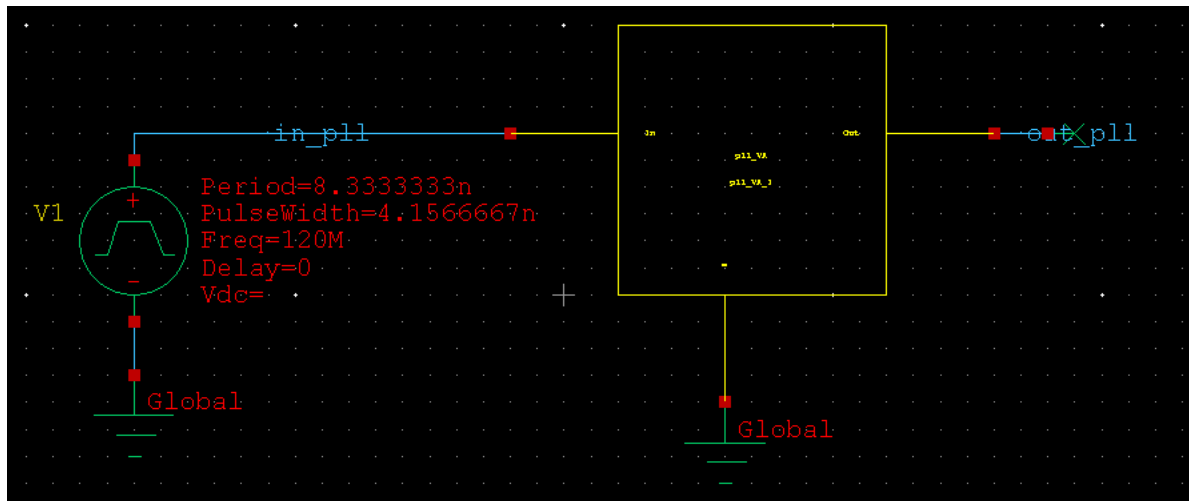
10. Transistor level divider testbench and results:



11. Integrate Verilog-A and simulate (show vco control voltage):



Testbench:



12. Using kvco get control voltage:

$$(VCTRL-0.2)*(2.1\text{GHz/V})+500\text{MHz}=960\text{MHz}$$

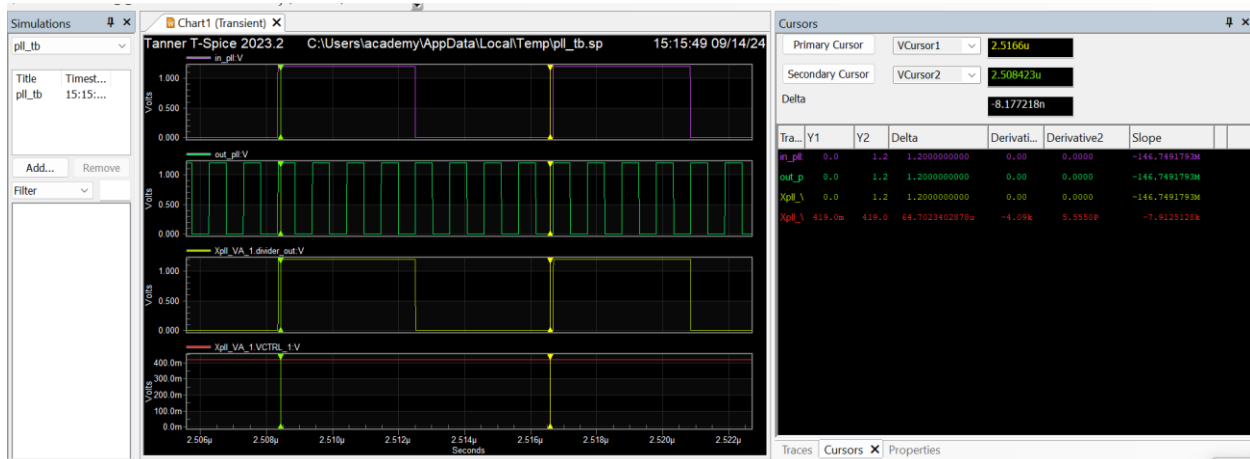
$$VCTRL=0.419047\text{V}$$

13. Compare the control voltage simulated value to the analytical value:

	Simulated	Analytical
Control voltage	419.049mV	419.047mV

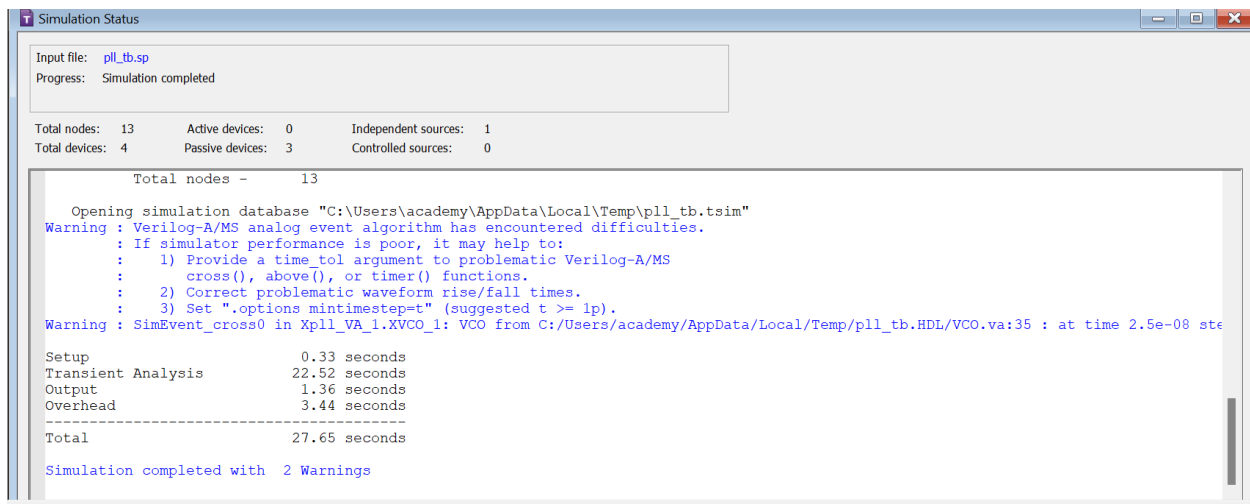
Comment: The results agree as expected with minor differences due to how simulation is done.

14. Integrate Verilog-A and simulate (illustrate operation):

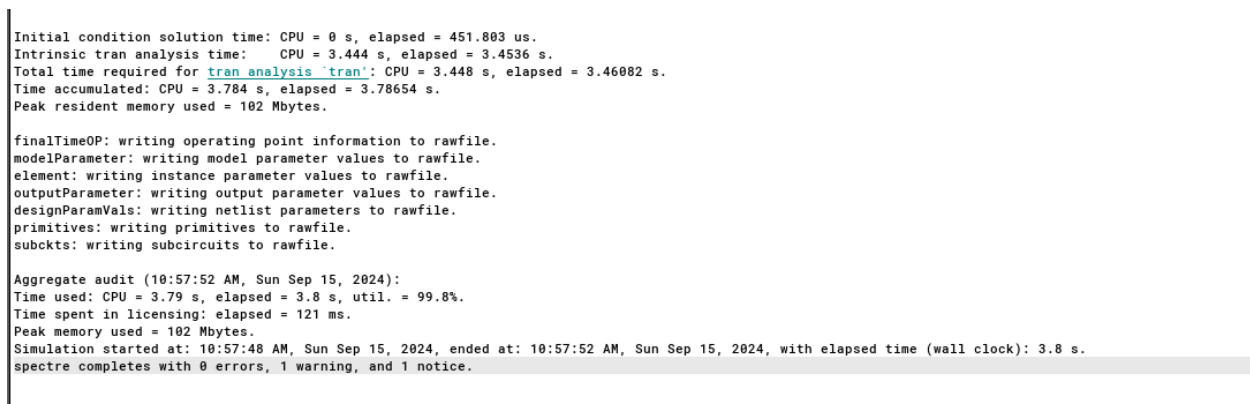


15. Snapshot of log file showing simulation time for Verilog-A:

Tanner:



Cadence:



16. Hierarchy editor:

Virtuoso® Hierarchy Editor Editing: (PLL_SIEMENS pll_tb config)

File Edit View Plugins Help

Search

Top Cell

Library: PLL_SIEMENS

Cell: pll_tb

View: pll_tb

Open Edit ADE L

Global Bindings

Library List: myLib

View List: atic veriloga ahdl pspice

Stop List: spectre

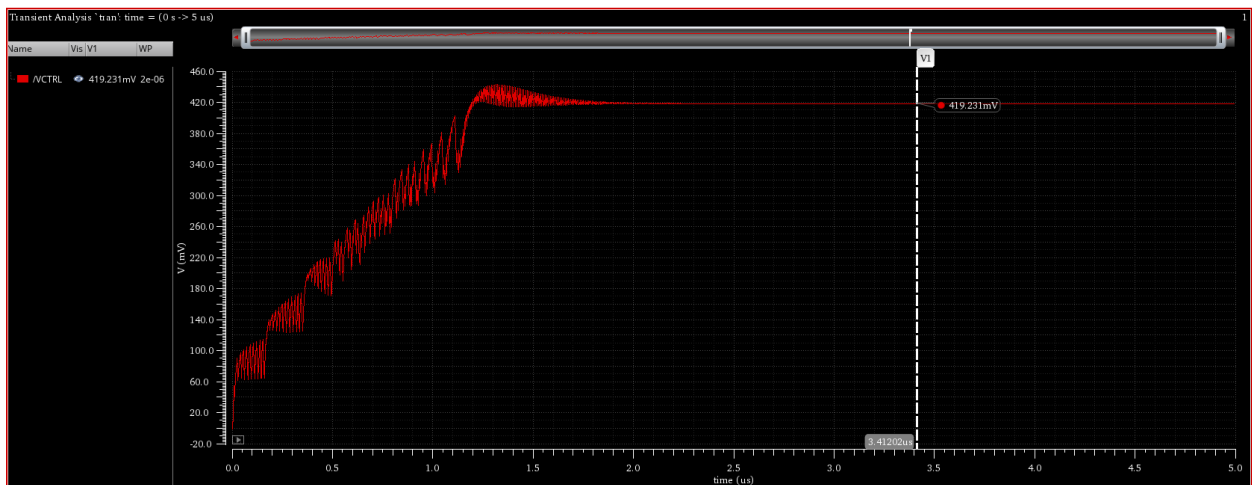
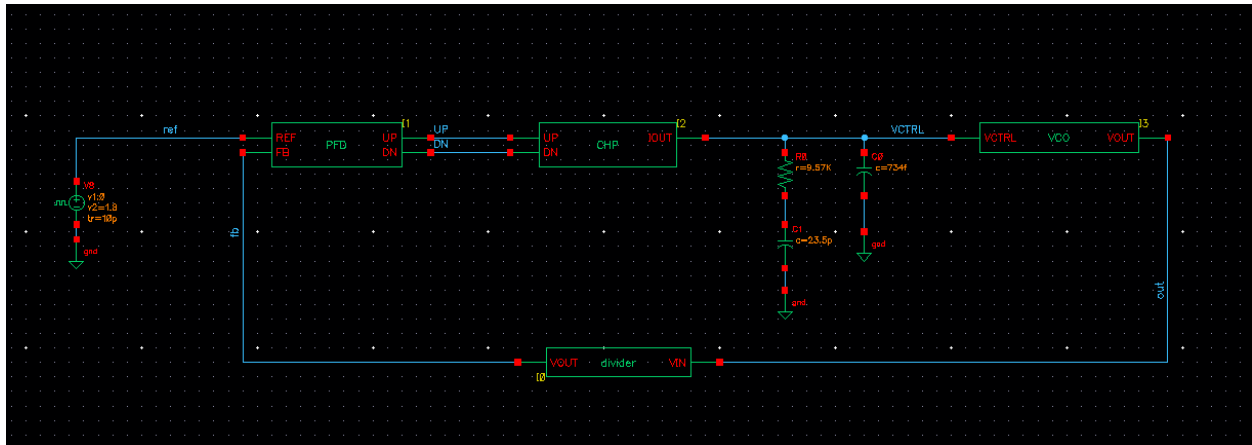
Constraint List:

Table View Tree View

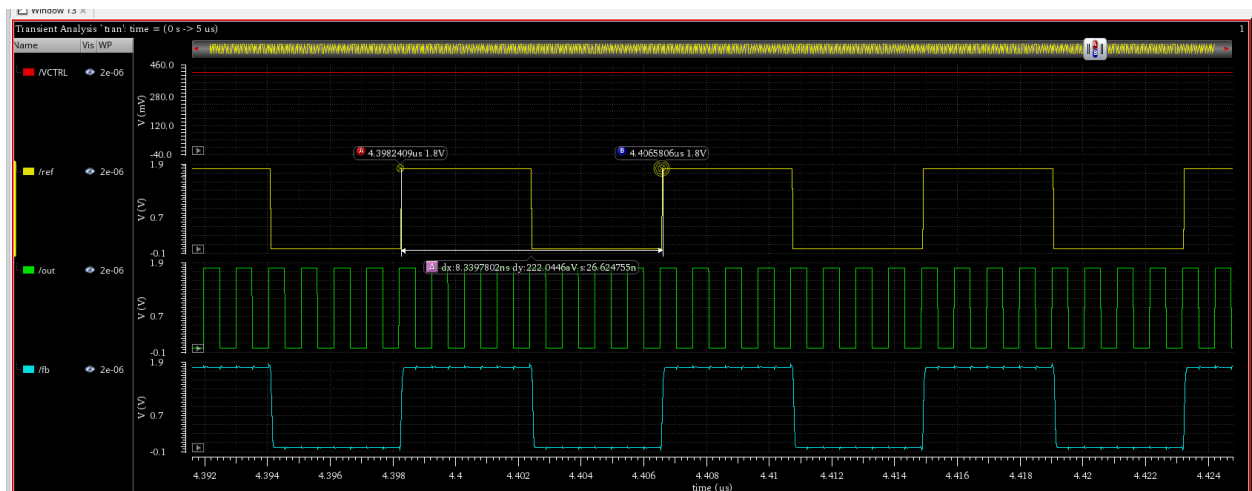
Cell Bindings

Library	Cell	View Found	View To Use	Inherited View List
PLL_SIEMENS	CHP	veriloga		spectre cmos_sch c...
PLL_SIEMENS	FF	schematic		spectre cmos_sch c...
PLL_SIEMENS	PFD	veriloga		spectre cmos_sch c...
PLL_SIEMENS	TG	schematic		spectre cmos_sch c...
PLL_SIEMENS	VCO	veriloga		spectre cmos_sch c...
PLL_SIEMENS	divider	schematic	schematic	spectre cmos_sch c...
PLL_SIEMENS	inv	schematic		spectre cmos_sch c...
PLL_SIEMENS	pll_tb	pll_tb		spectre cmos_sch c...
analogLib	cap	spectre		spectre cmos_sch c...
analogLib	nmos4	spectre		spectre cmos_sch c...
analogLib	pmos4	spectre		spectre cmos_sch c...
analogLib	res	spectre		spectre cmos_sch c...
analogLib	vdc	spectre		spectre cmos_sch c...
analogLib	vpulse	spectre		spectre cmos_sch c...

17. Integrate the PLL with using Verilog-A models for all blocks except for divider show VCO control voltage:



18. Integrate PLL with using Verilog-A models for all blocks except for divider, illustrate PLL operation:



19. Simulation log of PLL with divider in transistor level:

```
finalTimeOP: writing operating point information to rawfile.
modelParameter: writing model parameter values to rawfile.
element: writing instance parameter values to rawfile.
outputParameter: writing output parameter values to rawfile.
designParamVals: writing netlist parameters to rawfile.
primitives: writing primitives to rawfile.
subckts: writing subcircuits to rawfile.

Aggregate audit (10:54:41 AM, Sun Sep 15, 2024):
Time used: CPU = 148 s (2m 28.3s), elapsed = 149 s (2m 28.5s), util. = 99.8%.
Time spent in licensing: elapsed = 107 ms.
Peak memory used = 240 Mbytes.
Simulation started at: 10:52:13 AM, Sun Sep 15, 2024, ended at: 10:54:41 AM, Sun Sep 15, 2024, with elapsed time (wall clock): 149 s (2m 28.5s).
spectre completes with 0 errors, 1 warning, and 9 notices.
```

20. Compare simulation time and comment:

	Tanner T-SPICE(Verilog-A)	spectre (Verilog-A)	spectre (divider in schematic)
Simulation time	27.65s	3.8s	149s

Comment: the difference between T-SPICE and spectre in Verilog-A is likely due to different simulation settings and different computers. While the difference between the PLL with divider in schematic and Verilog-A is significant especially in comparison to spectre since it is the same simulation settings.